

Universidad Nacional Autónoma de México

Facultad de Ciencias

Cómputo Concurrente

2027-1

Práctica 02. Exclusión Mutua: Locks y Pools en Java

ALUMNOs:

Martínez Hidalgo Paola Mildred - 319300217
Dominguez Sanchez Gustavo Alonso - 418047121

Fecha de entrega:
10 de septiembre de 2026

Profesor:
Gilde Valeria Rodríguez



Ejercicios

1. El programa `ColaSecuencial` es una implementación de una cola secuencial. Implementa una Cola Concurrente utilizando una pool de hilos (`ExecutorService`). No utilices candados ni `synchronized`.

Implementación de `ColaConcurrente`: Análogo a la a `ColaSecuencial`, nuestro programa `ColaConcurrente` maneja la misma clase `Nodo` y utiliza la misma lógica para los métodos `enq()` (encolar) y `deq()` (desencolar) para poder visualizar mejor la comparación entre ambos programas. Tal como se pide: no usamos `synchronized`, ni la interfaz `Lock` (candados); por lo que utilizamos una pool de hilos, en particular, utilizando `ExecutorService` .
Para eliminar las condiciones de carrera, se creó una pool de de un solo hilo (`Executors.newFixedThreadPool(1)`), así que sin importar cuantos hilos llamen a `enq()` o `deq()`, el pool de hilos procesara estas tareas una por una.
Por otro lado para manejar las secciónes críticas (CS) creamos objetos `Callable`, que mediante funciones lambda, fue posible encapsular la abstracción de estas tareas, y poder pasarlas como parametro a `executor.submit()`. Mientras estas tareas son completadas, estas tareas pendientes son objetos de la clase `Future`. Una vez que el pool haya procesado la tarea, se nos devuelve un `boolean` (en el caso de `enq()`) y un `String` (en el caso de `deq()`).
Por último, como buena práctica, se añadió el método `shutdown()` para terminar la ejecución del pool (`ExecutorService`) correctamente, de lo contrario, sus hilos seguirán vivos esperando por nuevas tareas.

2. Ejecuta varias veces tu implementación con diferentes secuencias de llamadas a métodos ¿Tu implementación funciona de acuerdo a lo que se espera de una cola o suceden inconsistencias? Por ejemplo,

[i)]

- Se hace `enq(a)`, `enq(b)` y `deq()` y al final ambos elementos `a` y `b` están en la cola.
- Se realiza un `enq(a)` y un `enq(b)` y la cola solo contiene al elemento `a` o `b`.
- Se desencola dos veces el mismo elemento.

Si existe una ejecución así toma una captura pantalla del resultado de tu programa que lo ejemplifique.
Hint: Para analizar si hay inconsistencias, utiliza la interfaz Future, solo ten cuidado con el método get() ya que fuerza a que el resultado del future sea devuelto en ese momento.
Importante: El orden en el que se suben las tareas con submit y se guardan en los futures, no es el orden en el que se realizan, solo es el orden en el que se les asignan a los hilos. Nuestro sistema es asíncrono, es decir, cada hilo puede tardarse más o menos en ejecutar la tarea que se le asignó.

Implementación de `PruebaColaConcurrente`: Al analizar esta implementación, no suceden inconsistencias. EL programa funciona tal como se espera de para una cola (FIFO), esto ya que como se mencionó, hay un único hilo del `ExecutorService` dentro de `ColaConcurrente` que trabaja cada modificación a la cola, una por una.
El programa `PruebaColaConcurrente` simula el entorno de concurrencia que se busca, con el fin de encontrar algún error relacionado a las *Race Conditions* o *Data Race*. Para las pruebas, utilizamos un `ExecutorService` interno que cree las peticiones concurrentes a la `ColaConcurrente`, además guardamos los `Future` en una lista para no usar `get()` de manera inmediata, ya que de lo contrario, si llamaramos a `get()` justo después de cada `submit()` para obtener nuestros elementos, esto haria que el programa tenga que esperar a que termine cada tarea volviendolo secuencial.
Por lo que una vez creadas las peticiones concurrentes y guardar los future en las listas, simulamos un tiempo de espera `Thread.sleep(500)`, para darle tiempo a que el executor termine todas las peticiones. Después de de esta espera, ahora si obtenemos los `get()`, e imprimimos el estado de la cola para verificar que no tiene inconsistencias.

```
--- Iniciando pruebas de cola concurrente ---

Estado de la cola después de 10 enq() concurrentes:
Print
hnull
Element-2
Element-1
Element-3
Element-4
Element-5
Element-6
Element-7
Element-8
Element-9
Element-10

Elementos desencolados (deq):
Element-1
Element-5
Element-3
Element-2
Element-4

Estado final de la cola:
Print
hnull
Element-6
Element-7
Element-8
Element-9
Element-10
```

Figura 1: Una ejecución correcta y sin inconsistencias de `PruebaColaConcurrente.java`

```
--- Iniciando pruebas de cola concurrente ---

Estado de la cola después de 10 enq() concurrentes:
Print
hnull
Element-1
Element-2
Element-6
Element-7
Element-4
Element-3
Element-8
Element-5
Element-9
Element-10

Elementos desencolados (deq):
Element-2
Element-4
Element-6
Element-1
Element-7

Estado final de la cola:
Print
hnull
Element-3
Element-8
Element-5
Element-9
Element-10
```

Figura 2: Otra ejecución correcta y sin inconsistencias de `PruebaColaConcurrente.java`

3. ¿Existen data races o race-conditions? Explica en qué variables y porqué suceden.

En el programa original (`ColaConcurrente`), si existen tanto data races como race conditions:

- **Data Races**: Recordemos que esto ocurre cuando uno o más hilos acceden a un mismo recurso al mismo tiempo y al menos uno de ellos realiza una operación de escritura. Las variable que sufren de este problema son los punteros a los nodos (`this.head.next` y `this.tail.next`). Esto sucede tanto en:
  - `enq()`: Cuando dos o más hilos intentan encolar un elemento, leen la variable compartida `this.head.next` para louego ejecutar la líena `this.head.next = newnode`. Al intentar escribir los dos al mismo tiempo, se produce el data race.
  - `deq()`: Cuando dos o más hilos intentan desencolar, leen la variable compartida `this.head.next` y modifican la misma con `this.head.next = first.next`, produciendo el data race.
- `Race Conditions`: Recordemos que esta condición causa que el resultado del programa depende de la orden de la ejecución de los hilos por acceder al mismo tiempo a un recurso compartido. Esto ocurre en:
  - `enq()`: Suponiendo que la cola este vacía, cuando dos o más hilos realizan la lectura de `if(this.head.next == this.tail)`, para los demás hilos el resultado será `true`, sin embargo cuando un hilo escriba este primer nodo, los otros lo sobreescribirán, causando que en cada nueva ejecución, el estado de la cola sea distinta.

- `deq()`: Si la cola tiene elementos, y dos o más hilos ejecutan el método al mismo tiempo, terminaran ejecutando la linea `Nodo first = this.head.next` causando que desencolen solo un elemento, a pesar de que el método fue llamado dos o más veces.

4. Utiliza candados y/o `synchronized` en tu implementación de forma que los métodos `enq()` y `deq()` sean una sola seccción crítica y contesta: ¿existen inconsistencias?
Importante: El orden en el que se suben las tareas con submit y se guardan en los futures, no es el orden en el que se realizan, solo es el orden en el que se les asignan a los hilos. Nuestro sistema es asíncrono, es decir, cada hilo puede tardarse más o menos en ejecutar la tarea que se le asignó.

Implementación de `ColaConcurrenteSynchronized()`: Se nos pide que tanto el método `enq()` y `deq()` sean una sola sección crítica, es decir, que un solo hilo pueda acceder a la vez. Únicamente utilizaremos la palabra reservada `synchronized` en ambos métodos, que como ya vimos, al aplicar esto a diferentes métodos, este actuará como un "candado general", convirtiendo a todos estos métodos en una única sección crítica compartida.
Al garantizar exclusión mutua en ambos métodos, esto elimina las race conditions y las data race. Por lo que a nuestro parecer no hay inconsistencias.

```
--- Iniciando pruebas de cola concurrente ---

Estado de la cola después de 10 enq() concurrentes:
Print
hnull
Element-1
Element-3
Element-2
Element-4
Element-5
Element-6
Element-7
Element-8
Element-9
Element-10

Elementos desencolados (deq):
Element-1
Element-3
Element-2
Element-4
Element-5

Estado final de la cola:
Print
hnull
Element-6
Element-7
Element-8
Element-9
Element-10
```

Figura 3: Una ejecución correcta y sin inconsistencias de `PruebaColaConcurrente.java` utilizando `ColaConcurrenteSynchronized`

```
--- Iniciando pruebas de cola concurrente ---

Estado de la cola después de 10 enq() concurrentes:
Print
hnull
Element-3
Element-1
Element-2
Element-4
Element-5
Element-6
Element-7
Element-8
Element-9
Element-10

Elementos desencolados (deq):
Element-3
Element-1
Element-2
Element-4
Element-5

Estado final de la cola:
Print
hnull
Element-6
Element-7
Element-8
Element-9
Element-10
```

Figura 4: Otra ejecución correcta y sin inconsistencias de `PruebaColaConcurrente.java` utilizando `ColaConcurrenteSynchronized`

5. En una pool de hilos (`ExecutorService`), ¿si utilizamos más hilos equivale a un mayor *throughput*? Argumenta porqué.
No, utilizar más hilos no significa necesariamente a un mayor *throughput*. Esto lo vimos desde la práctica pasada, en donde la acelaración de un programa concurrente estará limitada por su proporción que es obligatoriamente secuencial, por lo que aumentar el número de hilos terminará costando más caro que el programa secuencial. Además, si las hilos comparten recursos y estos están tinenen exclusión mutua (tal como hicimos para `ColaConcurrente` y `ColaConcurrenteSynchronized`), los hilos perderán su tiempo esperando su turno, por lo que entre más hilos haya, mayor será el tiempo que pierde el sistema gestionandolos.

6. **Problema**. Supón que perteneces a un equipo de trabajo en el cual estás a cargo de dar acceso a un servidor. Seis personas te pueden mandar tareas para que las ejecutes en el servidor. Las tareas del hilo 0 y 2 tardan 500ns, las del hilo 1 tardan 2000ns y las de los hilos 3, 4 y 5 tardan 3000ns.
Tienes la instrucción de no dar acceso a más de tres al mismo tiempo y de no aceptar al mismo tiempo las tareas del hilo 0 y 2. Diseña e implementa una solución.
Hint: Apóyate del programa *Scheduler* y *Tarea*, el semáforo inicializado en 3 representa que solo pueden entrar 3 al mismo tiempo, decide como utilizarlo en el programa **Tarea** (colocando sus métodos `adquire` y `release`); además, decide como utilizar un candado, para restringir aceptar las tareas de los hilos 0 y 2 al mismo tiempo.

Para resolver el problema del servidor se modificaron las clases **Tarea** y **Scheduler**. En **Scheduler** se declaró un único objeto `Semaphore smphre = new Semaphore(3)`, inicializado en 3, que representa el máximo de tareas que pueden ejecutarse simultáneamente en el servidor; este mismo objeto se pasa por constructor a cada instancia de **Tarea** para garantizar que todas compartan el mismo semáforo. En **Tarea** se agregó un candado `Lock lock = new ReentrantLock()`, donde su propósito es impedir que las tareas provenientes de los hilos 0 y 2 se ejecuten al mismo tiempo entre sí. Dentro del método `run()`, toda tarea adquiere primero el permiso del semáforo (`smphre.acquire()`) antes de entrar a su sección crítica, si el valor calculado del hilo (`value`) es 0 o 2, entra el candado (`lock.lock()`) antes de ejecutar el `Thread.sleep()`. Las tareas con `value` igual a 1, 3, 4 o 5 solo interactúan con el semáforo, sin pasar nunca por el candado.

- ¿Tu implementación cumple con Justicia?

  En este caso, tal como está implementado actualmente, no se cumple. El programa no promete ningún orden, cuando dos o más tareas están esperando el pase y se libera uno Java puede dárselo a la tarea que "llegó después" si por casualidad su hilo está listo para tomarlo en ese instante, en lugar de respetar el orden de llegada. Por lo tanto, no cumple con justicia.

- Sino es así, describe cómo podrías garantizarla

  Para esto se tendría que usar la otra versión del constructor, tanto del semáforo como del candado, agregando el parámetro **true**:

  ```
  new Semaphore(3, true)
  new ReentrantLock(true)
  ```

  Este parámetro le indica a Java que debe cumplir estrictamente con la justicia, es decir, que cada tarea obtiene el permiso en el mismo orden en que lo solicitó, sin permitir que una tarea recién llegada se meta antes que otra que ya estaba esperando.